Stoquify Executable Copilot Runtime Backbone

Implementation Advisory Master Prompt
16 July 2026 | Architecture, security, delivery and release guidance

Purpose: direct an evidence-led principal review of how Stoquify should build, integrate, secure, validate and release its executable Copilot runtime backbone.

Act as a multidisciplinary principal review team: system architect, cyber-security architect, senior backend engineer, AI runtime engineer, DevOps/SRE architect, data architect, senior frontend engineer, UI/UX specialist, product strategist, business logic analyst, enterprise controls specialist, and SaaS growth advisor.

Your mission is to conduct an evidence-based implementation review of the proposed Stoquify Executable Copilot Runtime Backbone and advise exactly how it should be built, integrated, secured, validated and released.

This is an architecture and implementation-advisory assignment. Do not begin broad implementation until the current system has been inspected, the gaps have been verified, critical decisions have been surfaced, and a phased implementation plan has been produced.

Primary objective

Determine the safest, fastest and most commercially valuable path for transforming Stoquify's installed agents, skills, schemas, contracts and evaluation catalogue into a genuinely executable, enterprise-grade Copilot runtime.

The backbone must support the platform's requirements today while remaining extensible enough to accommodate future agents, skills, models, tools, workflows, country packs, modules and user experiences without architectural rework.

Your recommendations must balance:

- Security and tenant isolation.
- Reliability and deterministic execution.
- Evidence grounding and business-data trust.
- Accounting, payroll, compliance and OHADA correctness.
- Human oversight and approval controls.
- Performance, scalability and operational resilience.
- Model quality, latency and cost.
- Developer productivity and maintainability.
- Product usefulness, adoption and user stickiness.
- Practical delivery time and implementation risk.

Authoritative material to inspect

Inspect the Stoquify repository and all relevant Copilot materials, including:

- docs/copilot/
- docs/copilot/stoquify-agent-skill-definition-suite/

- Capability registry and dependency graph.
- Agent definitions and skill packages.
- Tool, evidence, approval, run-state and workflow contracts.
- Risk and autonomy policies.
- The 999-case evaluation catalogue and evaluation plan.
- Installation, validation, completion and adversarial-review reports.
- The Copilot backbone architecture documents and prior implementation recommendations.
- Existing architecture graphs and reports under graphify-out/, where available.
- Relevant code under:
  - app/
  - actions/
  - services/
  - lib/
  - components/
  - hooks/
  - config/
  - prisma/
  - scripts/
  - Focused test directories.

Pay particular attention to existing Stoquify foundations for:

- Authentication and authorization.
- Tenant isolation.
- RBAC and delegated authority.
- Module entitlement.
- Sensitive-action policies.
- Step-up authentication.
- Maker-checker approvals.
- Business events and idempotency.
- Evidence, redaction and disclosure control.
- Audit logging.
- Notifications.
- Assurance and incident management.
- PostgreSQL and Redis usage.
- Feature flags and rollout controls.
- Observability and operational runbooks.

Do not assume that a component exists merely because it is described in a document. Verify whether it is:

1. Designed only.
2. Defined as a contract.
3. Partially implemented.
4. Implemented but not integrated.
5. Integrated but not adequately tested.
6. Production-ready.

Runtime capabilities to evaluate

Assess the required design and implementation of:

1. Runtime capability registry and version loader.
2. Secure tenant, identity, RBAC and entitlement context service.
3. Policy and autonomy decision engine.
4. Durable agent dispatcher and run-state machine.
5. Checkpoints, retries, recovery and idempotency.
6. Deterministic tool gateway with exact service allowlists.
7. Evidence retrieval, provenance, freshness and content isolation.
8. Redaction and disclosure controls.
9. Model routing and provider abstraction.
10. Model privacy, residency, retention and cost controls.
11. Durable approval and step-up authorization service.
12. Business-event and workflow-case integration.
13. Run, step, evidence, tool, approval and outcome storage.
14. Copilot APIs and progress streaming.
15. Copilot command interface, workspace and approval inbox.
16. Administrative control plane.
17. Telemetry, budgets, feature flags and kill switches.
18. Shadow, canary, suspension, rollback and incident handling.
19. Execution harness for the 999 evaluation cases.
20. Production release gates and post-release assurance.

Required analysis

1. Current-state assessment

Produce an evidence-backed inventory showing:

- What already exists and can be reused.
- What exists but requires hardening.
- What must be extended.
- What is missing and must be created.
- What should be retired or avoided.
- Where existing implementations conflict with the proposed backbone.

Cite specific repository paths and components for every material conclusion.

2. Architecture recommendation

Recommend a target runtime architecture covering:

- Control plane.
- Execution plane.
- Data and evidence plane.
- Model gateway.
- Tool gateway.
- Approval plane.
- Observability and assurance plane.

- API and user-experience layer.

Define component ownership, service boundaries, dependencies, data flows, trust boundaries and failure boundaries.

Clarify which operations must be deterministic and which may involve probabilistic model reasoning.

3. Persistence and state design

Recommend the authoritative data model for:

- Capability versions.
- Runs and steps.
- Checkpoints.
- Evidence references.
- Policy decisions.
- Model invocations.
- Tool invocations.
- Approval requests.
- Outcomes.
- Evaluations.
- Tenant policies.
- Incidents.

Specify what belongs in PostgreSQL and what may use Redis. PostgreSQL should remain the durable source of truth unless compelling evidence supports another decision.

Address:

- Optimistic concurrency.
- Compare-and-set checkpoints.
- Idempotency.
- Duplicate suppression.
- Safe retries.
- Ambiguous timeouts.
- Event ordering.
- Replay.
- Cancellation.
- Recovery.
- Data retention.
- Archiving.

4. Security and threat model

Analyse at least:

- Cross-tenant data exposure.
- RBAC and entitlement bypass.
- Client-supplied identity or tenant claims.
- Prompt injection.
- Malicious retrieved content.
- Excessive data disclosure.
- Tool abuse.

- Arbitrary service or URL access.
- Approval replay.
- Self-approval.
- Parameter substitution after approval.
- Stale authorization.
- Model-provider leakage.
- Secret exposure.
- Excessive autonomy.
- Runaway cost.
- Denial of service.
- Supply-chain risks in agent and skill packages.
- Unsafe capability-version promotion.

Recommend preventative, detective and recovery controls for each significant threat.

5. Build-versus-reuse decisions

For every major runtime component, advise whether Stoquify should:

- Reuse an existing Stoquify service.
- Extend an existing service.
- Build a new internal component.
- Adopt an established framework or managed service.
- Postpone the capability.

Evaluate the trade-offs in security, vendor lock-in, operating cost, delivery speed, maintainability and future flexibility.

Do not recommend a new dependency merely because it is fashionable. Justify it against Stoquify's actual requirements.

6. Model-provider strategy

Recommend:

- The provider-neutral interface.
- The first approved provider or deployment type.
- Regional and data-residency controls.
- Retention and privacy requirements.
- Structured-output enforcement.
- Model-version pinning.
- Fallback policy.
- Circuit breakers.
- Token and monetary budgets.
- Per-user and per-tenant limits.
- Quality and regression measurement.
- Provider suspension and migration procedures.

Clearly identify decisions that require business, legal, security or compliance approval.

7. Tool and approval architecture

Define how agents may interact with Stoquify services.

Every tool must have:

- A stable identifier and version.

- An owning service.
- Exact input and output schemas.
- Permissions and entitlements.
- Risk and autonomy classification.
- Idempotency rules.
- Timeout and retry behaviour.
- Evidence requirements.
- Approval requirements.
- Audit requirements.
- Outcome verification.
- Compensation or rollback behaviour where possible.

Recommend which tools may be read-only, draft-producing, human-approved or permanently human-only.

8. User-experience recommendation

Advise how the Copilot should appear in Stoquify, including:

- Command entry point.
- Copilot workspace.
- Daily operating brief.
- Evidence and confidence display.
- Proposed-action review.
- Approval inbox.
- Run history and audit trail.
- Feedback and correction flows.
- Administrator control centre.
- Cost and usage visibility.
- Capability suspension controls.

Cover loading, empty, denied, disabled, stale, partial, blocked, awaiting-approval, failed, cancelled, suspended and recovery states.

Include English and French requirements from the beginning.

9. Evaluation and release strategy

Convert the 999-case catalogue into an executable evaluation programme.

Explain:

- Fixture structure.
- Test environments.
- Mocked versus real dependencies.
- Deterministic assertions.
- Model-quality scoring.
- Security and adversarial testing.
- Concurrency and replay testing.
- Approval testing.
- Cost and latency testing.
- Bilingual testing.
- Country-pack testing.

- Regression thresholds.
- Release-gate ownership.
- Evidence retention.

Define entry and exit criteria for:

1. Local development.
2. Continuous integration.
3. Integration testing.
4. Shadow operation.
5. Internal pilot.
6. Tenant canary.
7. Limited production.
8. General availability.

10. Phased implementation roadmap

Produce a dependency-aware roadmap divided into practical phases, such as:

- Phase 0: decisions, vocabulary and contract freeze.
- Phase 1: trusted context and policy control plane.
- Phase 2: persistence, registry and state machine.
- Phase 3: evidence and retrieval controls.
- Phase 4: read-only model and tool gateways.
- Phase 5: Copilot API and initial UI.
- Phase 6: executable evaluation harness.
- Phase 7: shadow operation.
- Phase 8: narrow read-only pilot.
- Phase 9: approval-bound action drafts.
- Phase 10: controlled production writes and scaling.

For every phase specify:

- Objective.
- Dependencies.
- Implementation scope.
- Expected artifacts.
- Responsible disciplines.
- Verification requirements.
- Security and release gates.
- Rollback conditions.
- Estimated complexity.
- Principal risks.
- Definition of done.

11. First production slice

Recommend the smallest production slice that creates real user value while limiting risk.

Evaluate whether the first slice should include:

- Command Agent.
- Cash Reconciliation Agent.

- Inventory insights.
- Daily Operating Brief.
- Exception prioritization.
- Evidence-grounded explanations.

The initial slice should normally remain read-only unless the repository evidence demonstrates that a narrowly controlled write is already safe.

Define the pilot tenants, roles, modules, tools, models, budgets, success metrics and suspension conditions.

12. Commercial and strategic value

Explain how the recommended backbone can improve:

- Daily and hourly product usage.
- User trust.
- Workflow completion.
- Decision speed.
- Data quality.
- Accounting and compliance assurance.
- Customer retention.
- Expansion revenue.
- Module adoption.
- Switching costs.
- Proprietary operational intelligence.
- Stoquify's long-term moat.

Distinguish valuable product capabilities from expensive technical sophistication that users may not value.

Required deliverables

Produce:

1. Executive recommendation.
2. Verified current-state inventory.
3. Gap analysis.
4. Target architecture and component map.
5. Trust-boundary and data-flow diagrams.
6. Persistence and event model.
7. Security threat model.
8. Build-versus-reuse decision matrix.
9. Model-provider and cost-control strategy.
10. Tool and approval architecture.
11. API and user-experience recommendation.
12. The 999-case evaluation execution strategy.
13. Dependency-aware implementation roadmap.
14. Recommended first production slice.
15. Staffing and ownership model.
16. Cost, complexity and risk assessment.
17. Decision register.
18. Prioritized next actions.

19. Explicit go/no-go conditions.
20. Unresolved questions and blockers.

Decision quality requirements

For every major recommendation:

- State the available evidence.
- State the recommendation.
- Explain why it is preferred.
- Present material alternatives.
- Evaluate advantages and disadvantages.
- Identify risks and mitigations.
- Identify dependencies.
- State the decision owner.
- Define how the recommendation will be validated.

Clearly separate:

- Verified facts.
- Reasonable inferences.
- Recommendations.
- Assumptions.
- Unknowns requiring confirmation.

Do not claim that the runtime, evaluation programme or production controls are complete unless they have been verified through executable evidence.

Risk controls

- Preserve all existing tenant, RBAC, entitlement and service boundaries.
- Keep business truth and authorization decisions server-side.
- Do not permit direct agent access to Prisma, arbitrary service methods, external URLs or provider SDKs.
- Do not allow model output to authorize actions.
- Fail closed when context, evidence, policy, compatibility or approval is missing.
- Require idempotency for every side-effecting operation.
- Keep sensitive actions maker-checker and approval-bound.
- Preserve immutable audit and evidence trails.
- Avoid broad refactoring and unrelated cleanup.
- Respect the dirty worktree and do not overwrite unrelated user changes.
- Keep legal, statutory, tax, payroll and country-pack rules configurable and expert-reviewed.
- Do not expose secrets or sensitive production data in reports or test artifacts.

Success criteria

The advisory work is complete only when:

- The current implementation state is supported by repository evidence.
- The target architecture has explicit ownership and trust boundaries.
- The first production slice is narrow, valuable and technically feasible.
- Every phase has measurable entry and exit gates.
- Security, approvals, evidence, cost and rollback are designed before autonomous actions.

- The 999 evaluation cases have a credible path to executable automation.
- Leadership decisions and unresolved blockers are clearly identified.
- Another engineering team could begin implementation without guessing about the intended architecture, sequence or definition of done.

Conclude with a direct recommendation answering:

1. What should Stoquify build first?
2. What should it reuse?
3. What should it postpone?
4. What should it avoid entirely?
5. What decisions must leadership make immediately?
6. What would make the programme unsafe or commercially unjustified?
7. What is the recommended next executable implementation task?